Employee Asset Assignment Register

Technical Requirement Document - Odoo 18.0

Module: Employee Asset Assignment Register

Odoo Version: 18.0

Module Display Name: Employee Asset Register

Module Technical Name: `md_employee_asset_register`

Target Users: HR / Admin / IT Team

1. Objective

Develop a dedicated Odoo module to maintain an internal employee asset register and track asset assignment to employees.

The module must allow authorized users to:

- Create and manage asset records.
- Assign assets to employees.
- Track asset status and condition.
- View linked assets directly from the employee form.
- Maintain a central list of all assets with filters, grouping, and search

options.

- Track business-critical changes through chatter.

This module is intended only for operational asset assignment tracking.

2. Scope

2.1 In Scope

The module must provide:

1. A new asset register model.
2. Asset category configuration.
3. Asset assignment to employees.
4. Asset status tracking.
5. Asset condition tracking.
6. One2many tab on the employee form showing linked assets.
7. Dedicated asset menus and actions under the Employees app.
8. List, form, search, and kanban views.
9. Default filters and groupings.
10. Chatter tracking for business-critical fields.
11. Security group and access rights.

12. Project-compliant manifest, module layout, naming, translation, and app description files.

2.2 Out of Scope

The module must not handle:

1. Purchase orders.
2. Vendor bills.
3. Inventory stock moves.
4. Stock valuation.
5. Accounting assets.
6. Depreciation.
7. Barcode scanning.
8. Asset request approvals.
9. Employee portal access.
10. Automated procurement or replacement flow.
11. Client-side bridge APIs.
12. API keys, credentials, or external integrations.

3. Development Standards

Mandatory standards for this module:

- Technical module name must start with `md_`.
- Module folder name must exactly match `md_employee_asset_register`.
- New custom model names, XML IDs, view names, action IDs, menu IDs, and security IDs must contain `md`.
- Custom field technical names must start with `md_`.
- Standard Odoo fields must not be renamed with the `md_` prefix.
- User-facing labels must be clear, human-readable, and Title Case.
- Python files must start with `# -*- coding: utf-8 -*-`.
- New project model classes must use `Md...` class names.
- Each Python model file must contain only one model.
- Every custom model must have at least one required field.
- Manifest author must be `mindphin`.
- Manifest website must be `www.mindphin.com`.
- Manifest license must be `OPL-1`, unless a different license is approved.
- Security access rights must be defined for every custom model.
- Required i18n files must be included:
- `i18n/de_CH.po`
- `i18n/fr_CH.po`
- `i18n/it_IT.po`

- Static app description files must be included when the module is visible in the Apps list.

- `README.md` must document purpose, scope, installation, and exclusions.

- No debug code, unused imports, commented-out code, secrets, archives, or `__pycache__/` folders may be committed.

4. Module Structure

Required module directory:

```
md_employee_asset_register/
■■■ __init__.py
■■■ __manifest__.py
■■■ README.md
■■■ data/
■   ■■■ ir_sequence_data.xml
■   ■■■ md_employee_asset_category_data.xml
■■■ i18n/
■   ■■■ de_CH.po
■   ■■■ fr_CH.po
■   ■■■ it_IT.po
■■■ models/
■   ■■■ __init__.py
■   ■■■ hr_employee.py
■   ■■■ md_employee_asset.py
■   ■■■ md_employee_asset_category.py
■■■ security/
■   ■■■ ir.model.access.csv
■   ■■■ md_employee_asset_register_groups.xml
■■■ static/
■   ■■■ description/
■       ■■■ icon.png
■       ■■■ index.html
■■■ views/
    ■■■ hr_employee_views.xml
    ■■■ md_employee_asset_category_views.xml
    ■■■ md_employee_asset_views.xml
    ■■■ menus.xml
```

Data load order in the manifest must be:

1. Security files.
2. Data files.
3. Views.
4. Menus last.

5. Dependencies

The module must depend only on modules it uses:

```
"depends": [
    "hr",
    "mail",
]
```

Reasons:

- `hr` is required to link assets with employees, departments, and the employee form.

- `mail` is required for chatter and activity tracking.

6. Manifest Requirements

Suggested manifest values:

```
# -*- coding: utf-8 -*-

{
    "name": "Employee Asset Register",
    "version": "18.0.1.0.0",
    "author": "mindphin",
    "website": "www.mindphin.com",
    "category": "Human Resources",
    "summary": "Track internal assets assigned to employees.",
    "description": """
Employee Asset Register

Track internal operational assets and their assignment to employees.
This module does not manage purchasing, inventory valuation, accounting assets,
depreciation, procurement, or portal workflows.
""",
    "license": "OPL-1",
    "depends": ["hr", "mail"],
    "data": [
        "security/md_employee_asset_register_groups.xml",
        "security/ir.model.access.csv",
        "data/ir_sequence_data.xml",
        "data/md_employee_asset_category_data.xml",
        "views/md_employee_asset_category_views.xml",
        "views/md_employee_asset_views.xml",
        "views/hr_employee_views.xml",
        "views/menus.xml",
    ],
    "demo": [],
    "images": ["static/description/icon.png"],
    "installable": True,
    "application": False,
    "auto_install": False,
}
```

7. Model: Employee Asset

7.1 Model Details

Property	Value
Model Name	`md.employee.asset`
Description	Employee Asset
Python File	`models/md_employee_asset.py`
Python Class	`MdEmployeeAsset`
Inherits	`mail.thread`, `mail.activity.mixin`
Rec Name	`name`
Ordering	`md_asset_code asc, id desc`

7.2 Fields: `md.employee.asset`

Standard Odoo field names must remain unprefixed where applicable. Custom fields must use the `md_` prefix.

Field Label	Technical Name	Type	Required	Tracking	Notes
Asset Name	`name`	Char	Yes	Yes	Name of the asset
Asset Code	`md_asset_code`	Char	Yes	Yes	Unique internal code
Category	`md_category_id`	Many2one -> `md.employee.asset.category`	Yes	Yes	Asset category

Field Label	Technical Name	Type	Required	Tracking	Notes
Brand	`md_brand`	Char	No	No	Brand name
Model	`md_model`	Char	No	No	Model name or number
Assigned Employee	`md_employee_id`	Many2one -> `hr.employee`	No	Yes	Currently assigned employee
Department	`md_department_id`	Related Many2one -> `hr.department`	No	No	Related from employee
Assigned Date	`md_assigned_date`	Date	No	Yes	Date of assignment
Return Date	`md_return_date`	Date	No	Yes	Date of return
Status	`state`	Selection	Yes	Yes	Standard status field name
Condition	`md_condition`	Selection	Yes	Yes	Current asset condition
Location	`md_location`	Char	No	No	Current physical location
Notes	`md_notes`	Text	No	No	Internal remarks
Active	`active`	Boolean	No	No	Standard active field, default `True`

7.3 Computed / Related Fields

Field Label	Technical Name	Type	Store	Notes
Department	`md_department_id`	Related Many2one	Yes	Related to `md_employee_id.department_id`
Display Label	`md_display_label`	Computed Char	No	Format: `[Asset Code] Asset Name`, if needed for display

Suggested display format:

```
[AST-00001] Wired Mouse
```

The implementation must not expose technical field names such as `md_asset_code` in the UI.

8. Selection Values

8.1 Status Field: `state`

```
[
    ("in_stock", "In Stock"),
    ("assigned", "Assigned"),
    ("returned", "Returned"),
    ("damaged", "Damaged"),
    ("lost", "Lost"),
    ("scrapped", "Scrapped"),
]
```

Default value:

```
"in_stock"
```

8.2 Condition Field: `md_condition`

```
[
    ("new", "New"),
    ("good", "Good"),
    ("used", "Used"),
    ("damaged", "Damaged"),
]
```

Default value:

```
"new"
```

9. Asset Code Sequence

9.1 Requirement

Asset Code must be auto-generated when creating an asset if the user does not manually enter one.

Default format:

```
AST-00001
AST-00002
AST-00003
```

9.2 Sequence XML

Create the sequence in `data/ir_sequence_data.xml`.

Suggested sequence:

```
<record id="md_employee_asset_sequence" model="ir.sequence">
    <field name="name">Employee Asset</field>
    <field name="code">md.employee.asset</field>
    <field name="prefix">AST-</field>
    <field name="padding">5</field>
    <field name="number_next">1</field>
    <field name="number_increment">1</field>
</record>
```

9.3 Developer Notes

Override `create()` only for sequence assignment:

- If `md_asset_code` is empty or `/`, assign the next sequence.
- `md_asset_code` must be unique.
- Add a concise comment explaining the override purpose.
- Validation messages must use `_()` for translation readiness.

10. Constraints and Validations

10.1 Unique Asset Code

Add SQL constraint:

```
_sql_constraints = [
    (
        "md_employee_asset_code_unique",
        "unique(md_asset_code)",
        "Asset Code must be unique.",
    ),
]
```

10.2 Assignment Validation

Validation rules:

1. If `state` is `assigned`, `md_employee_id` must be selected.
2. If `state` is `assigned`, `md_assigned_date` must be set.
3. `md_return_date` cannot be earlier than `md_assigned_date`.

Required validation messages:

```
Assigned Employee is required when Status is Assigned.
Assigned Date is required when Status is Assigned.
```

```
Return Date cannot be earlier than Assigned Date.
```

Messages must be implemented with `_()`.

11. Business Logic

11.1 Onchange Employee

When `md_employee_id` is selected:

- If `md_assigned_date` is empty, set it to today's date using `fields.Date.context_today(self)`.
- Set `state` to `assigned`.

11.2 Onchange State

When `state` changes to `returned`:

- If `md_return_date` is empty, set it to today's date using `fields.Date.context_today(self)`.

When `state` changes to `damaged`:

- Set `md_condition` to `damaged`.

12. Buttons

Add object buttons on the asset form header.

12.1 Button: Mark Assigned

Visible when `state != "assigned"`.

Method:

```
action_mark_assigned()
```

Behavior:

- Validate that `md_employee_id` is selected.
- Set `state` to `assigned`.
- Set `md_assigned_date` to today if empty.

12.2 Button: Mark Returned

Visible when `state == "assigned"`.

Method:

```
action_mark_returned()
```

Behavior:

- Set `state` to `returned`.
- Set `md_return_date` to today if empty.

12.3 Button: Mark Damaged

Visible when `state != "damaged"`.

Method:

```
action_mark_damaged()
```

Behavior:

- Set `state` to `damaged`.
- Set `md_condition` to `damaged`.

12.4 Button: Mark Lost

Visible when `state != "lost"`.

Method:

`action_mark_lost()`

Behavior:

- Set `state` to `lost`.

12.5 Button: Scrap

Visible when `state != "scrapped"`.

Method:

`action_scrap()`

Behavior:

- Set `state` to `scrapped`.
- Do not automatically archive the record in version 1.

13. Model: Employee Asset Category

13.1 Model Details

Property	Value
Model Name	`md.employee.asset.category`
Description	Employee Asset Category
Python File	`models/md_employee_asset_category.py`
Python Class	`MdEmployeeAssetCategory`
Rec Name	`name`
Ordering	`sequence, name`

13.2 Fields: `md.employee.asset.category`

Field Label	Technical Name	Type	Required	Notes
Name	`name`	Char	Yes	Category name
Code	`md_code`	Char	No	Short category code
Sequence	`sequence`	Integer	No	Standard sequence field, default `10`
Active	`active`	Boolean	No	Standard active field, default `True`

14. Default Asset Categories

Create default category records in `data/md_employee_asset_category_data.xml`.

Category	Code
Mouse	MOU
Keyboard	KBD
Monitor	MON
Laptop Charger	CHG
Headset	HDS
Adapter	ADP
Laptop Stand	STD
Other	OTH

All XML IDs must contain `md`, for example:

```
<record id="md_employee_asset_category_mouse" model="md.employee.asset.category">
    <field name="name">Mouse</field>
    <field name="md_code">MOU</field>
</record>
```

15. Employee Model Extension

15.1 Model

Inherit:

```
hr.employee
```

Python file:

```
models/hr_employee.py
```

15.2 Fields

Add One2many field:

Field Label	Technical Name	Type	Related Model	Inverse Field
Assigned Assets	`md_asset_ids`	One2many	`md.employee.asset`	`md_employee_id`

Add computed count field:

Field Label	Technical Name	Type	Notes
Asset Count	`md_asset_count`	Integer	Count assigned assets

The count must include only assets where:

```
md_employee_id = current employee
state = assigned
```

16. Views

All view XML IDs and technical view names must contain `md`.

16.1 Asset List View

XML ID:

```
md_employee_asset_view_list
```

Columns:

```
Asset Code
Asset Name
```

```
Category
Brand
Model
Assigned Employee
Department
Assigned Date
Status
Condition
Location
```

Recommended decorations:

Condition	Decoration
`state == "assigned"`	Success
`state == "damaged"`	Warning
`state == "lost"`	Danger
`state == "scrapped"`	Muted

16.2 Asset Form View

XML ID:

```
md_employee_asset_view_form
```

Header:

- Statusbar for `state`.
- Object buttons:
- Mark Assigned
- Mark Returned
- Mark Damaged
- Mark Lost
- Scrap

Main sheet:

- Show `active` with `widget="boolean_toggle"` at the top-right if visible.
- Group fields logically.

Asset Information:

```
Asset Code
Asset Name
Category
Brand
Model
Condition
Location
```

Assignment Information:

```
Assigned Employee
Department
Assigned Date
Return Date
Status
```

Notes:

- Place `md_notes` under a `Notes` separator.
- Use sufficient width and `colspan="2"` where applicable.

Chatter must be enabled below the form.

16.3 Asset Search View

XML ID:

```
md_employee_asset_view_search
```

Filters:

```
Assigned
In Stock
Returned
Damaged
Lost
Scrapped
Archived
```

Group By:

```
Status
Category
Assigned Employee
Department
Condition
Brand
Location
```

Default group by:

```
Status
```

16.4 Asset Kanban View

XML ID:

```
md_employee_asset_view_kanban
```

Kanban cards should show:

```
Asset Code
Asset Name
Category
Assigned Employee
Status
Condition
```

Default grouping:

```
Status
```

16.5 Asset Category Views

Required XML IDs:

```
md_employee_asset_category_view_list
md_employee_asset_category_view_form
md_employee_asset_category_view_search
```

List view columns:

```
Sequence
Name
Code
Active
```

The `sequence` field must use `widget="handle"` in the list view.

16.6 Employee Form View Extension

Inherited view XML ID:

```
md_hr_employee_view_form_inherit_assets
```

Inherited technical view name:

```
md.hr.employee.view.form.inherit.assets
```

Add a notebook page on the Employee form:

```
Assigned Assets
```

Inside the page, add the One2many field `md_asset_ids`.

Columns in the One2many list:

```
Asset Code
Asset Name
Category
Brand
Model
Assigned Date
Status
Condition
```

The One2many must show all asset records linked to the employee.

17. Menus and Actions

Create menus under the Employees app:

```
Employees
■■■ Asset Register
    ■■■ Assets
    ■■■ Asset Categories
```

Required action XML IDs:

```
md_employee_asset_action
md_employee_asset_category_action
```

Required menu XML IDs:

```
md_employee_asset_menu_root
md_employee_asset_menu
md_employee_asset_category_menu
```

Menu details:

Menu	Action
Asset Register	Parent menu
Assets	Opens `md.employee.asset`
Asset Categories	Opens `md.employee.asset.category`

Only users in the Employee Asset Manager group must see these menus.

18. Security

18.1 User Group

Create one security group:

```
Employee Asset Manager
```

Technical XML ID:

```
md_group_employee_asset_manager
```

Suggested category:

```
Human Resources
```

Admin auto-assignment must be reviewed as required by the Mindphin security standards.

18.2 Access Rights

Access CSV IDs must follow the Mindphin pattern `access_md_<model>_<role>`.

`md.employee.asset`

Group	Read	Create	Write	Delete
Employee Asset Manager	Yes	Yes	Yes	Yes

`md.employee.asset.category`

Group	Read	Create	Write	Delete
Employee Asset Manager	Yes	Yes	Yes	Yes

Suggested access IDs:

```
access_md_employee_asset_manager
access_md_employee_asset_category_manager
```

For version 1:

- Only users in the Employee Asset Manager group should access the asset menus.
- Normal employees should not see the asset menu.

19. Record Rules

For version 1, do not create restrictive record rules.

Access should be controlled by:

- Security group.
- Menu visibility.
- Access rights.

Optional future rules, not required in version 1:

- Employees can read their own assigned assets.
- Department managers can read department assets.

20. Chatter Tracking

Enable chatter on `md.employee.asset`.

Track these fields:

```
Asset Code
Asset Name
Category
Assigned Employee
Assigned Date
Return Date
Status
Condition
```

Expected chatter behavior:

```
Status changed from In Stock to Assigned.
Assigned Employee changed.
Condition changed from New to Damaged.
```

Only fields with clear business audit value should be tracked.

21. Translation Readiness

All user-facing dynamic messages must use `_()`.

Required translation files:

```
i18n/de_CH.po
i18n/fr_CH.po
i18n/it_IT.po
```

Visible strings requiring translation readiness include:

- Field labels.
- Button labels.
- Menu labels.
- Action names.
- View page names.
- Validation messages.
- Help text and descriptions.

22. Static Description and README

The module must include:

```
static/description/icon.png
static/description/index.html
README.md
```

README must document:

- What the module does.
- What the module does not do.
- Installation steps.
- Configuration steps.
- Security group usage.
- Relation to other Mindphin modules, if applicable.

23. Sample Records

Sample records are for documentation or demo data only. Do not hardcode employee names, department names, company names, credentials, or instance-specific data.

23.1 Assigned Asset

```
Asset Code: AST-00001
Asset Name: Wired Mouse
Category: Mouse
Brand: Sample Brand
Model: Sample Model
Assigned Employee: Demo Employee
Department: Demo Department
Assigned Date: 02 July 2026
Status: Assigned
Condition: New
Location: Office
```

23.2 Unassigned Asset

```
Asset Code: AST-00002
Asset Name: Keyboard
Category: Keyboard
Brand: Sample Brand
Model: Sample Model
Assigned Employee: Empty
Department: Empty
Assigned Date: Empty
Status: In Stock
Condition: New
Location: Store Room
```

24. Acceptance Criteria

24.1 Asset Register

- User can create an asset.
- Asset Code is mandatory and unique.
- Asset Code is auto-generated if not entered.
- User can select category, brand, model, condition, and status.
- User can assign an asset to an employee.
- All custom field names follow `md_` naming rules.

24.2 Employee Integration

- Employee form has a tab called Assigned Assets.
- The tab shows assets linked to that employee.
- Assigned asset count is available if implemented as a smart button or computed field.
- Employee extension fields use `md_` technical names.

24.3 Workflow

- User can mark an asset as Assigned.
- User can mark an asset as Returned.
- User can mark an asset as Damaged.
- User can mark an asset as Lost.
- User can mark an asset as Scrapped.
- State changes are tracked in chatter.
- Validation messages are translatable.

24.4 Views

- Asset list view is available.
- Asset form view is available.
- Asset kanban view is available.
- Asset search view has filters and group-by options.
- Asset menu is available under Employees.
- Asset category menu is available.
- XML IDs, action IDs, menu IDs, and view names contain `md`.

24.5 Security

- Only authorized users can access the asset menus.
- Normal employees should not see the asset menu unless access is explicitly granted.
- Access rights exist for both custom models.
- Security IDs follow Mindphin naming standards.

24.6 Mindphin Compliance

- Module folder and technical module name are `md_employee_asset_register`.
- Manifest uses `author: mindphin`.
- Manifest uses `website: www.mindphin.com`.
- Manifest uses `license: OPL-1`, unless otherwise approved.
- Required `i18n` files exist.
- Static description assets exist.
- README exists.
- No unrelated dependencies are introduced.
- No secrets, debug code, temporary code, archives, or generated cache files are committed.

25. Development Notes

25.1 Python Guidelines

Use:

```
from odoo import _, api, fields, models
from odoo.exceptions import ValidationError
```

Implementation requirements:

- Use `@api.constrains` for validations.
- Use `fields.Date.context_today(self)` for today's date.
- Use sequence inside `create()` only for asset code generation.
- Use `_()` for all validation messages.
- Avoid `sudo()` unless justified and documented.
- Avoid hardcoded employee names, company names, asset-specific values, secrets, API keys, and instance-specific data.
- Remove unused imports.
- Do not use `print()` for server-side logging.

25.2 XML Guidelines

Use Odoo 18 XML view syntax.

Required XML IDs:

```
md_employee_asset_view_list
md_employee_asset_view_form
md_employee_asset_view_search
md_employee_asset_view_kanban
md_employee_asset_category_view_list
md_employee_asset_category_view_form
md_employee_asset_category_view_search
md_hr_employee_view_form_inherit_assets
md_employee_asset_action
md_employee_asset_category_action
md_employee_asset_menu_root
md_employee_asset_menu
md_employee_asset_category_menu
```

Inherited view technical names must include `_inherit` or `_inh`.

25.3 Git and Review

Repository changes must stay scoped to this module.

Suggested branch:

```
feature/md_employee_asset_register
```

Suggested commit message:

```
[ADD][VSC] Add employee asset register module
```

26. Future Enhancements

The following items are not part of version 1:

1. QR code / barcode label printing.
2. Asset handover report.
3. Employee exit clearance checklist.
4. Asset request and approval workflow.
5. Import template for bulk asset creation.
6. Warranty expiry reminder.
7. Asset assignment history model.
8. Employee portal visibility.
9. Accounting asset integration.
10. Bulk assign / unassign wizard.
11. Asset replacement workflow.

27. Final Implementation Summary

The developer should create a dedicated module named:

```
md_employee_asset_register
```

The module should provide:

- Central asset register.
- Asset categories.
- Employee assignment.
- Employee form One2many tab.
- Status and condition tracking.
- Chatter tracking.
- Sequence-based asset code.
- Menu under Employees.
- Access rights for authorized users.
- Project-compliant naming, manifest, security, translation, README, and static description files.

The solution must remain lightweight and focused on operational asset assignment tracking.